Code Splitting & Lazy Loading Routes

5 min read Updated 24 Mar 2026 Topic 19 of 62

3.3 - Code Splitting & Lazy Loading Routes

🎯 What Interviewers Actually Ask

Question	Level	Frequency
"What is code splitting?"	🟡	★★★★☆
"What is lazy loading in React?"	🟡	★★★★☆
"What is React.lazy and Suspense?"	🟡	★★★★☆
"How do you implement route-based code splitting?"	🟡🔴	★★★☆☆
"What is the difference between lazy loading and code splitting?"	🟡	★★★☆☆

The Problem: Bundle Size

When you build a React app, the bundler (Webpack, Vite) combines all your JavaScript into one or a few large files. For a small app, this is fine. For a large app with dozens of pages, the user downloads the code for every page upfront, even pages they may never visit.

Code splitting breaks your bundle into smaller chunks. **Lazy loading** loads those chunks only when they're needed (usually when the user navigates to that route).

They're related but distinct concepts. Code splitting is the technique of breaking the bundle apart. Lazy loading is the strategy of loading those pieces on demand.

React.lazy and Suspense

`React.lazy` lets you define a component that is loaded dynamically. It takes a function that returns a dynamic `import()`.

`Suspense` wraps lazy components and shows a fallback UI while the component is loading.

JSX Copy

```jsx
import { lazy, Suspense } from 'react';

// Instead of: import Dashboard from './Dashboard';
const Dashboard = lazy(() => import('./Dashboard'));
const Settings = lazy(() => import('./Settings'));
const Analytics = lazy(() => import('./Analytics'));

function App() {
  return (
    <BrowserRouter>
      <Suspense fallback={<div>Loading ... </div>}>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/dashboard" element={<Dashboard />} />
          <Route path="/settings" element={<Settings />} />
          <Route path="/analytics" element={<Analytics />} />
        </Routes>
      </Suspense>
```

When the user navigates to `/dashboard`, React downloads the Dashboard chunk, shows "Loading..." while it downloads, then renders the component. The Settings and Analytics code is never downloaded until the user visits those pages.

Where to Place Suspense

You can wrap individual routes or the entire Routes block. Each approach has tradeoffs:

JSX Copy

```jsx
// Option A: Single Suspense around all routes
// Simpler, but shows the same loading state for every route transition
<Suspense fallback={<PageLoader />}>
  <Routes>
    <Route path="/dashboard" element={<Dashboard />} />
    <Route path="/settings" element={<Settings />} />
  </Routes>
</Suspense>

// Option B: Suspense per route
// More code, but each route can have its own loading UI
<Routes>
  <Route
    path="/dashboard"
    element={
      <Suspense fallback={<DashboardSkeleton />}>
        <Dashboard />
      </Suspense>
    }
  />
  <Route
    path="/settings"
    element={
      <Suspense fallback={<SettingsSkeleton />}>
        <Settings />
      </Suspense>
```

Option B with skeleton screens gives a much better user experience because each page shows a loading state that matches its layout.

Combining with Protected Routes

Lazy loading and auth guards work together naturally:

JSX Copy

```jsx
const Dashboard = lazy(() => import('./pages/Dashboard'));
const AdminPanel = lazy(() => import('./pages/AdminPanel'));

function App() {
  return (
    <BrowserRouter>
      <AuthProvider>
        <Suspense fallback={<PageLoader />}>
          <Routes>
            <Route path="/login" element={<LoginPage />} />
            <Route
              path="/dashboard"
              element={
                <ProtectedRoute>
                  <Dashboard />
                </ProtectedRoute>
              }
            />
            <Route
              path="/admin"
              element={
                <RoleRoute allowedRoles={['admin']}>
                  <AdminPanel />
                </RoleRoute>
              }
            />
          </Routes>
```

```
    );
}
```

The AdminPanel code is never even downloaded unless the user navigates to `/admin` AND passes the role check.

Named Exports and Lazy

`React.lazy` only works with **default exports**. If a module uses named exports, you need a wrapper:

JSX Copy

```jsx
// ❌ Won't work with named export
const MyComponent = lazy(() => import('./MyComponent'));
// Error if MyComponent uses: export function MyComponent()

// ✅ Wrap named export in a default export re-assignment
const MyComponent = lazy(() =>
  import('./MyComponent').then(module => ({ default: module.MyComponent }))
);
```

⚡ Common Mistakes

Mistake 1: Lazy loading everything. Only lazy load routes (page-level components). Don't lazy load small, frequently used components like buttons or inputs. The overhead of loading them on demand is worse than including them in the main bundle.

Mistake 2: Forgetting Suspense. If a lazy component renders without a Suspense boundary above it, React throws an error. Always wrap lazy components in Suspense.

Mistake 3: No error handling for failed loads. If the network fails while loading a chunk, the user sees an uncaught error. Wrap the Suspense in an Error Boundary for production apps.

JSX Copy

```jsx
<ErrorBoundary fallback={<p>Failed to load page. Please refresh.</p>}>
  <Suspense fallback={<PageLoader />}>
    <Routes> ... </Routes>
  </Suspense>
</ErrorBoundary>
```

Interview Answer

"Code splitting breaks the JavaScript bundle into smaller chunks. Lazy loading loads those chunks on demand. In React, I use `React.lazy` with dynamic `import()` to define components that load only when their route is visited. `Suspense` provides a fallback UI while the chunk downloads. I apply this at the route level, wrapping each page in a Suspense with a skeleton loader. For production, I combine it with Error Boundaries to handle network failures gracefully. The key rule is to only lazy load page-level components, not small reusable ones."

Key Takeaways

- **Code splitting** breaks the bundle into smaller files. **Lazy loading** loads them on demand.
- `React.lazy(() => import('./Page'))` creates a dynamically loaded component.
- `Suspense` provides a fallback while the lazy component loads. Required for every lazy component.
- Apply lazy loading at the **route/page level**, not on small components.

- Combine with **Protected Routes** so unauthorized users never download restricted code.

📚 Module 3 Complete!

#	Article	Core Concept
3.1	React Router	URL mapping, hooks, nested routes, v5 vs v6
3.2	Protected Routes	Auth guards, role-based access, redirect flow
3.3	Code Splitting & Lazy Loading	React.lazy, Suspense, route-level splitting

✨ QUICK CHECK — no marks, just for you.

1. According to the lesson, what is the distinction between code splitting and lazy loading?

- ○ Code splitting is the technique of breaking the bundle apart; lazy loading is the strategy of loading those pieces on demand
- ○ They are two names for the same technique, one used by Webpack and one used by Vite
- ○ Code splitting loads chunks on demand; lazy loading decides at build time which files to emit
- ○ Code splitting is a browser feature; lazy loading is a bundler feature that replaces it

2. A teammate wraps every Button and Input component in the design system with React.lazy to 'shrink the bundle everywhere'. Per the lesson, why is this a mistake?

- ○ The overhead of loading small, frequently used components on demand is worse than just including them in the main bundle
- ○ Lazy components cannot accept props, so shared UI primitives break when lazified
- ○ Bundlers refuse to emit a chunk smaller than the main bundle, so nothing is actually split
- ○ React.lazy silently ignores components that render fewer than a certain number of DOM nodes

Check answer

3. Your lazy routes are correctly wrapped in a Suspense boundary with a PageLoader fallback. A user on a flaky connection navigates to /dashboard and the chunk request fails. What does the lesson say happens, and what fixes it?

- ○ The user sees an uncaught error; wrap the Suspense in an Error Boundary to handle it
- ○ The Suspense fallback is re-rendered as the error UI; customize the fallback prop to show a retry message
- ○ The Suspense fallback stays on screen forever; shorten the fallback timeout to recover
- ○ React.lazy automatically retries the import until it succeeds, so no fix is needed

Check answer

Protected Routes & Authentication Flow

Next

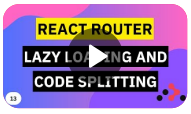
State Management: When & Why You Need It

LEARN MORE

ARTICLES & DOCS

- lazy — API reference
  react.dev
- Suspense — API reference
  react.dev

VIDEOS



Lazy loading & code splitting with React Router
YouTube



Code splitting in React — performance
YouTube



Route-based code splitting & lazy loading
YouTube

Hand-picked references for this lesson.